

Project: Gemini Accelerator Backend for ONNX-MLIR

Gemini Pipeline in ONNX-MLIR

From ONNX Model to RISC-V Binary and Gemini Execution

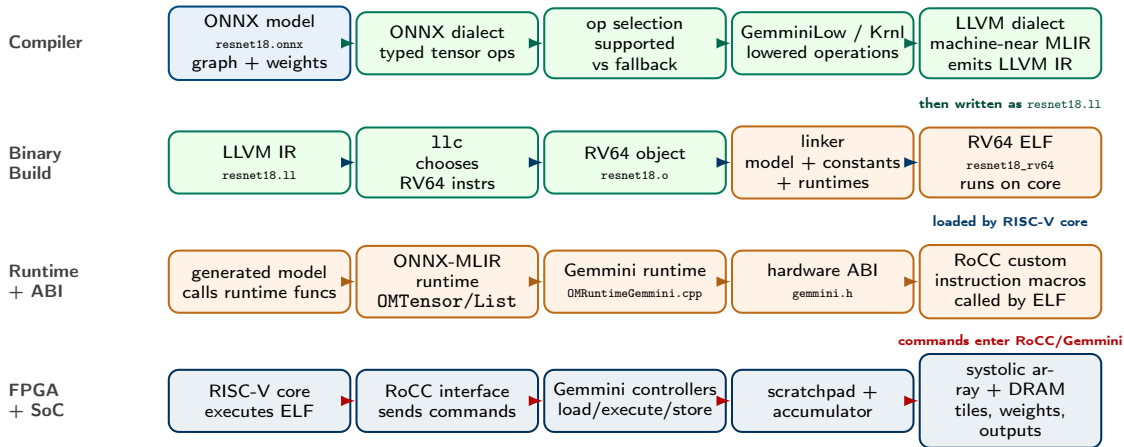
Siliconesignal Technologies

July 15, 2026

Big Picture

Start with the whole path before diving into files, functions, and ABI details.

How ONNX Moves to FPGA Execution



How to Read This Slide: green arrows = compiler lowering, blue arrows = binary build/link, orange arrows = runtime/ABI calls, red arrows = hardware command/data movement. Text boxes are separated by lanes to avoid mixing source files, compiler IR, runtime code, and FPGA blocks.

Pipeline Summary: Why Each Layer Exists

Layer	Why it exists	What it produces
Compiler IR	Keeps model meaning visible while choosing Gemini or fallback paths.	ONNX, Gemini, GeminiLow, Krnl, and LLVM dialect forms.
Binary build	Turns typed LLVM IR into RV64 machine code and links required runtimes.	Object files, linked ELF, constants, runtime symbols.
Runtime	Gives generated model code stable functions that understand tensor descriptors.	Calls such as <code>om_gemmini_conv_f32</code> .
ABI	Makes separately compiled code and hardware commands agree at binary level.	C calls, <code>gemmini.h</code> macros, RoCC instruction fields.
Hardware	Executes tiled matrix operations using scratchpad, accumulator, and systolic array.	Gemmini load, execute, and store behavior.

One sentence

The compiler decides and lowers; the runtime bridges tensors to helper code; the ABI keeps binary calls and hardware commands correct.

Who Uses ONNX-MLIR and Why It Matters

Context

ONNX-MLIR is not end-user software. It is a developer-facing compiler framework, so adoption is concentrated around hardware vendors, cloud/runtime builders, AI framework teams, and compiler researchers.

Group	Role in the ecosystem	Why it matters for this project
IBM	Origin and named adopter. The ONNX-MLIR README lists IBM zDLC using ONNX-MLIR technology for IBM Telum servers.	Shows the same compiler idea can target real accelerator-backed production hardware.
Microsoft + Meta	Core ONNX ecosystem leaders and ONNX standard originators; relevant to model interchange and framework compatibility.	ONNX-MLIR depends on stable ONNX semantics before any Gemini lowering can happen.
Arm / edge CPU ecosystem	Important target space for efficient neural-network lowering on CPUs and edge processors.	Reinforces why compiler IR, lowering, and hardware-aware code generation matter beyond one chip.
Serving + downstream hardware	Related serving projects and custom accelerator forks can use ONNX-MLIR as a pluggable frontend.	Gemini follows this pattern: reuse ONNX-MLIR frontend/compiler structure, then add accelerator-specific dialect/runtime/ABI.

Sources: [ONNX-MLIR GitHub README](#) | [ONNX-MLIR ecosystem talk](#) | [IBM Research talk](#) | [Arm/compiler talk](#)

Why This Project Exists

Goal

Compile neural-network models represented in ONNX into RISC-V executables that can use the Gemini matrix accelerator.

- ONNX expresses the model as graph operations such as Conv, MatMul, Add, Relu, and Pool.
- ONNX-MLIR translates the graph through progressively lower compiler IR levels.
- Gemini-specific passes identify operations that should run on the accelerator.
- The final binary contains normal RISC-V code plus runtime calls and Gemini custom instructions.

How the Final Binary Is Built



Important

The final RV64 executable is a linked program, not a one-to-one copy of the ONNX graph. It contains:

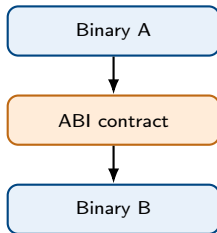
- generated model code for graph control, tensor setup, and calls;
- constants such as weights and bias tensors in read-only data;
- ONNX-MLIR runtime code for `OMTensor` and `OMTensorList`;
- Gemmini runtime code such as `om_gemmini_conv_f32`;
- Gemmini hardware ABI calls that become RISC-V custom instructions.

Each step removes abstraction: graph operation, compiler operation, runtime call, object symbol, linked address, and finally disassembled RISC-V instructions.

Core Terms

Define the vocabulary before using it repeatedly in compiler and runtime diagrams.

ABI Defines the Binary Contract

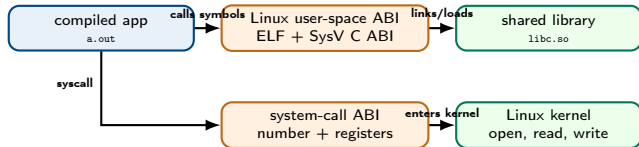


Application Binary Interface

Rules that let compiled code and binary components call each other correctly.

- Calling convention, registers, stack.
- Pointer sizes, alignment, data layout.
- Symbol names, relocation records, libraries.
- Linux example: ELF binary + libc + syscalls.
- Gemini example: runtime calls + hardware commands.

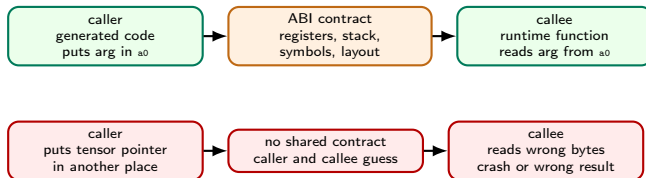
How Linux Uses an ABI



Concrete Linux Meaning

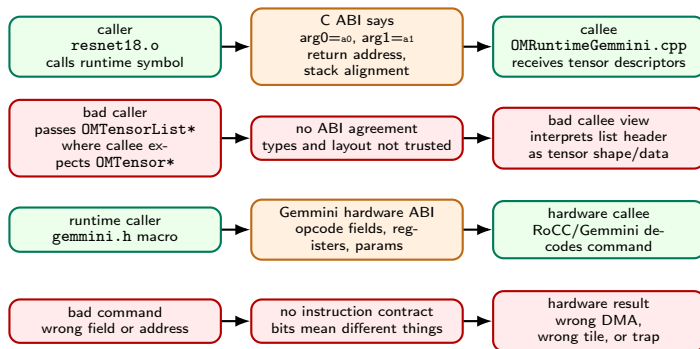
A Linux binary works because it follows agreed binary rules: ELF file format, symbol names, register usage for function calls, stack layout, syscall numbers, and which registers carry syscall arguments.

What Breaks Without an ABI?



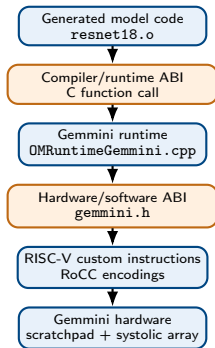
- The caller is the code that starts the call; in this project it can be generated model code.
- The callee is the code receiving the call; here it can be `om_gemmini_conv_f32`.
- Without an ABI, both sides may disagree on symbol names, registers, stack slots, and struct layout.
- The program may still link or run, but it can pass corrupted arguments to the runtime or hardware.

How Caller/Callee Mismatch Breaks Execution



Caller: the side that starts a call or sends a command, for example `resnet18.o` or a `gemmini.h` macro. **Callee:** the side that receives it, for example `OMRuntimeGemmini.cpp` or Gemmini hardware. The ABI is the shared promise that lets both sides understand the same call, data layout, and hardware command bits.

Where ABI Boundaries Appear in This Project



Key idea

There are two practical ABI boundaries: generated code follows a C ABI when it calls runtime functions, then runtime code follows the Gemmini hardware ABI when it issues accelerator commands.

Which Files Define the ABI Boundary

Compiler/runtime ABI

Generated model code calls functions declared in: `src/Accelerators/Gemmini/Runtime/OMRuntimeGemmini.hpp`

- Example function: `om_gemmini_conv_f32(out, x, w, stride, pad)`.
- Inputs and outputs cross this boundary as `OMTensor` * descriptors.
- The implementation lives in `OMRuntimeGemmini.cpp`.

Hardware/software ABI

Runtime code reaches Gemmini through:

`src/Accelerators/Gemmini/Runtime/gemmini-hardware-abi/include/gemmini.h`

- `gemmini.h` defines macros/helpers for Gemmini RoCC custom instructions.
- In disassembly, these hardware commands can appear as raw `.insn` encodings.

Remember These Terms: Compiler Pipeline

Term	Definition to remember
Compiler	Program that rewrites source/model representation into lower-level code until it becomes an object file or executable.
Compiler pass	One compiler step that analyzes or rewrites IR, such as converting ONNX Conv to Gemmini runtime calls.
Pass pipeline	Ordered list of passes; order matters because each pass expects a specific input form and produces the next form.
Dialect	MLIR namespace with its own operations, attributes, and types, such as <code>onnx</code> , <code>gemmini</code> , <code>gemminilow</code> , or LLVM dialect.
Conversion	Pattern-based rewrite from one dialect/form to another, for example ONNX to Gemmini or GemminiLow to LLVM.
Lowering	Rewriting the same computation closer to execution: model operation to runtime call, hardware step, or machine instruction.

Memory rule: compiler chooses the path, passes do the work, dialects name each level, conversions/lowering move between levels.

Remember These Terms: Runtime and Data

Term	Definition to remember
Runtime	Linked support code that executes inside the final binary between generated model code and low-level hardware or library calls.
ABI	Binary contract that tells caller and callee how to agree on symbols, registers, stack, data layout, and hardware instruction fields.
OMTensor	ONNX-MLIR tensor descriptor: points to tensor data and records shape, strides, rank, element type, and ownership metadata.
OMTensorList	Container of OMTensor * entries used for model inputs and outputs at the model boundary.
Gemmini runtime	Runtime functions such as <code>om_gemmini_conv_f32</code> that read tensors and call Gemmini helper/ABI code.
Hardware ABI	<code>gemmini.h</code> contract for encoding Gemmini/RoCC commands, parameters, registers, and generated hardware constants.

Memory rule: runtime executes, ABI defines the binary agreement, OMTensor describes one tensor, OMTensorList groups tensors.

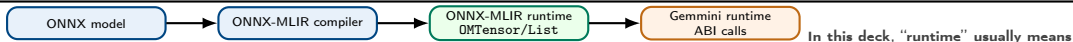
Remember These Terms: IR and Binary Build

Term	Definition to remember
Krnl	ONNX-MLIR internal dialect for kernel-like implementation details: loops, memory, constants, and calls before LLVM lowering.
memref	MLIR memory-reference type that describes shaped memory buffers before they become lower-level pointers.
LLVM IR	Typed compiler IR consumed by LLVM tools; in this project it contains calls such as <code>om_gemmini_conv_f32</code> .
Object file	Relocatable machine-code file such as <code>resnet18.o</code> ; symbols may still need the linker.
Linker	Tool that combines model object code, constants, ONNX-MLIR runtime, Gemmini runtime, and libraries into the final ELF.
ELF	Executable and Linkable Format; the final RV64 binary loaded by Spike or hardware.

Memory rule: Krnl/memref are still compiler-level implementation forms; object/linker/ELF are binary build forms.

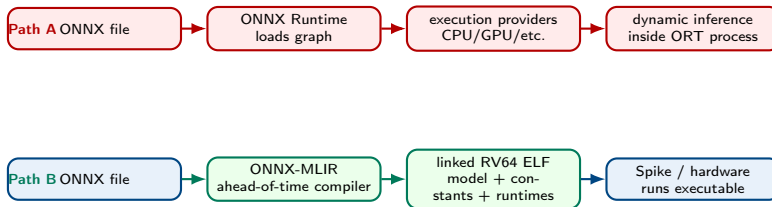
ONNX Runtime vs ONNX-MLIR Runtime

Term	What it is	Relation to this project
ONNX Runtime	A separate inference engine that loads and executes ONNX models directly.	Useful for normal ONNX inference, but it is not the runtime linked into the generated RV64 Gemini binary.
ONNX-MLIR Runtime	Runtime support library used by code produced by the ONNX-MLIR compiler.	Provides OMTensor, OMTensorList, entry/return conventions, and support code for compiled model execution.
Gemmini Runtime	Accelerator-specific runtime code under src/Accelerators/Gemmini/Runtime/.	Adds functions such as om_gemmini_conv_f32 and bridges ONNX-MLIR tensor descriptors to Gemini ABI calls.



ONNX-MLIR runtime plus Gemini runtime linked into the compiled executable, not Microsoft ONNX Runtime.

Why ONNX Runtime Is Not Used Here



Question	Answer for this project
Why not ONNX Runtime?	Because after compilation there is no ONNX graph to interpret at execution time. The model has already become RV64 code, constants, runtime calls, and Gemini custom instructions.
What replaces it?	The linked executable uses ONNX-MLIR runtime for <code>OMTensor/OMTensorList</code> and Gemini runtime for accelerator kernels.
What would ONNX Runtime do?	It would load the original <code>.onnx</code> model and dispatch nodes through its own execution-provider system. That is a different execution architecture from this AOT Gemini binary.

What Breaks If We Expect ONNX Runtime Here?

Confusion	Clear mental model
"The binary should call ONNX Runtime."	No. The binary calls generated functions and runtime helpers such as <code>om_gemmini_conv_f32</code> . ONNX Runtime APIs like <code>OrtSession</code> are not part of this RV64/Gemmini path.
"ONNX Runtime will choose Gemmini."	No. Gemmini selection happens during ONNX-MLIR lowering passes, before the final executable is linked.
"The <code>.onnx</code> file is needed at inference time."	No. The final executable already contains model code and constants. The ONNX file is an input to compilation, not an input to the deployed RV64 binary.
"Runtime means ONNX Runtime."	Not in this deck. Here, runtime means the small support code linked with compiled model code: ONNX-MLIR runtime plus Gemmini runtime.

Short rule

ONNX Runtime executes ONNX models dynamically. ONNX-MLIR produces an executable ahead of time. This project is the second path.

Utility of ONNX Runtime

Simple definition

ONNX Runtime is useful when the deployment goal is: load a `model.onnx` file at execution time, choose kernels dynamically, and run inference through an inference engine.

Utility	Meaning
Direct ONNX execution	It reads the ONNX graph at runtime and executes nodes without first producing a standalone compiler-generated executable.
Execution providers	It can dispatch work to CPU, CUDA, TensorRT, DirectML, and other provider backends depending on the installed platform.
Flexible deployment	The same <code>.onnx</code> model can be moved between machines and providers without rebuilding a target-specific binary each time.
Runtime services	It handles graph loading, operator scheduling, kernel dispatch, memory planning, and model session APIs.

`model.onnx` → ONNX Runtime → inference result

Can We Use ONNX Runtime in Our Gemmini Case?

Use	Yes / No	Explanation
Final RV64/Gemmini execution	No	The final executable does not load <code>model.onnx</code> ; it runs compiled RV64 code and calls ONNX-MLIR/Gemmini runtime functions.
CPU baseline	Yes	Run the same <code>resnet18.onnx</code> with ONNX Runtime on CPU and compare outputs or timing against the Gemmini binary.
Golden reference output	Yes	Use ONNX Runtime to generate expected output tensors, then check whether Gemmini output matches.
Model validity check	Yes	If ONNX-MLIR or Gemmini lowering fails, ONNX Runtime can confirm whether the original ONNX model itself is valid.
Direct Gemmini provider	Not currently	That would require a custom ONNX Runtime Execution Provider for Gemmini. This project instead uses ONNX-MLIR AOT compilation.

Compare ONNX Runtime and ONNX-MLIR Runtime

Point	ONNX Runtime	ONNX-MLIR Runtime
Main role	Runs <code>.onnx</code> models directly.	Supports code generated by ONNX-MLIR.
Execution style	Dynamic inference engine.	Linked support library for compiled model execution.
Inference input	<code>model.onnx</code> .	Compiled binary/shared library plus user input tensors.
Who reads graph?	ONNX Runtime reads the graph at runtime.	ONNX-MLIR compiler reads the graph before runtime.
Kernel choice	Runtime chooses kernels/providers.	Compiler/lowering already selected calls and code paths.
Gemmini path	Would need a Gemmini Execution Provider.	Uses generated code, Gemmini runtime, and Gemmini ABI.
Our project	Useful as CPU/reference baseline.	Required for OMTensor, OMTensorList, and compiled model support.

ONNX Runtime = dynamic ONNX inference

ONNX-MLIR Runtime = support for compiled ONNX

Why ONNX-MLIR Avoids Depending on ONNX Runtime

Reason	Explanation
Ahead-of-time compilation	ONNX-MLIR wants the ONNX graph compiled before execution, not interpreted at execution time.
Compiler optimizations	It can lower operations into loops, LLVM IR, target-specific calls, and accelerator instructions.
Smaller deployment dependency	The final program needs only compiled model code and support runtimes, not a full ONNX inference engine.
Hardware backend control	Compiler passes can target CPUs, IBM NNPA, Gemini, or other accelerators directly through lowering and ABI code.
Static embedded path	A standalone RV64 binary is easier to run on Spike, FPGA SoCs, or constrained systems than a dynamic ONNX Runtime stack.
Clear ABI	Generated code calls known functions such as <code>om_gemmini_conv_f32</code> ; hardware commands are emitted through <code>gemmini.h</code> .

Important

ONNX Runtime is not bad or useless globally. It is simply the wrong execution architecture for this AOT Gemini compiler pipeline.

Why Our Path Is Better for Gemini

Scope

This does not mean ONNX-MLIR is always better than ONNX Runtime. It means ONNX-MLIR is better for this Gemini/RV64 accelerator pipeline.

Advantage	Why it matters for Gemini
Direct hardware targeting	Conv and MatMul can lower into Gemini runtime calls, RoCC custom instructions, and Gemini ABI commands.
Ahead-of-time binary	Spike or hardware runs a normal RV64 ELF; it does not need to load or interpret an ONNX graph.
Compiler visibility	ONNX-MLIR can inspect, rewrite, lower, tile, and specialize operations before inference starts.
Smaller execution stack	The final path links only needed support code instead of carrying a full dynamic inference engine.
ABI control	We control the exact boundary from generated code to <code>om_gemmini_conv_f32</code> , then to <code>gemmini.h</code> custom instructions.
Research extensibility	New dialects, passes, lowering rules, and Gemini-specific optimizations can be added inside the compiler pipeline.

ONNX Runtime = flexible general inference engine
ONNX-MLIR + Gemini = specialized compiler path for hardware acceleration

When to Choose Each Path

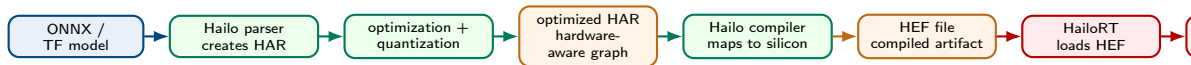
Choose	Best target environment	Why
ONNX Runtime + Custom EP	Cloud servers, PCs, and powerful edge systems such as Jetson-class devices.	RAM is usually not the main constraint, models may change frequently, and dynamic provider dispatch is useful.
ONNX-MLIR	Proprietary ASIC/FPGA accelerators, critical embedded systems, RISC-V SoCs, and IoT processors.	Every kilobyte matters, the target hardware is highly specific, and raw execution speed matters more than software flexibility.

Decision rule

ONNX Runtime is the strong default for standard cloud inference. ONNX-MLIR is preferred when the goal is to remove software overhead and produce maximum raw performance on highly specialized hardware.

Custom EP path = flexible engine
ONNX-MLIR path = compiled hardware-specific executable

Hailo Pipeline: AOT Compiler Suite Example

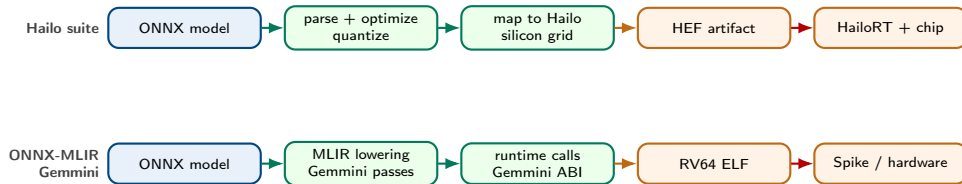


What this shows

Hailo uses an ahead-of-time compiler flow: the model is parsed, optimized, quantized, mapped onto the physical dataflow resources of the chip, and emitted as a fixed compiled artifact. Production inference loads the compiled artifact; it does not need ONNX Runtime to interpret the original graph.

Sources: [Hailo Developer Zone documentation](#) | [Hailo Model Zoo](#)

Hailo and ONNX-MLIR Share the Same Compiler Philosophy



Shared idea	Explanation
AOT compilation	Both paths move expensive graph analysis and hardware mapping before deployment.
No production ONNX graph interpreter	In production, Hailo loads HEF and ONNX-MLIR/Gemmini runs an ELF; neither path needs ONNX Runtime to interpret the original ONNX graph.
Hardware-specific artifact	Hailo emits an artifact for Hailo silicon; ONNX-MLIR/Gemmini emits code and ABI calls for RISC-V + Gemini.

Mirror Code: HailoRT vs ONNX-MLIR Runtime

HailoRT idea

Python loads a compiled **HEF** artifact, opens hardware streams, and pushes preprocessed tensors toward the Hailo NPU.



Equivalent ONNX-MLIR PyRuntime idea

Python loads a compiled **.so** artifact and calls native compiled model code through `OMExecutionSession`.

```
import numpy as np
import cv2
from PyRuntime import OMExecutionSession

SO_FILE = "my_compiled_model.so" # like Hailo .hef
IMAGE_FILE = "test_image.jpg"

session = OMExecutionSession(SO_FILE)

batch, channels, height, width = 1, 3, 224, 224
img = cv2.imread(IMAGE_FILE)
img = cv2.resize(img, (width, height))
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
img = img.transpose(2, 0, 1) # NHWC -> NCHW
input_data = np.ascontiguousarray(
    np.expand_dims(img, axis=0), dtype=np.float32)

outputs = session.run([input_data])
print(outputs[0].shape)
```

In both versions, Python is only the host-side plumbing: it prepares NumPy memory and sends it into a compiled black box. No runtime loop inspects the ONNX graph during inference.

What Changes Between HailoRT and ONNX-MLIR Runtime

Logical step	HailoRT inference on NPU	ONNX-MLIR Runtime inference on compiled .so
Binary artifact	<code>.hef</code> : a spatial dataflow configuration compiled for Hailo silicon.	<code>.so</code> / <code>.dll</code> : native machine-code library produced by ONNX-MLIR.
Hardware setup	<code>VDevice()</code> initializes the physical NPU through PCIe, USB, or the platform driver.	<code>OMExecutionSession()</code> loads the shared library into RAM and prepares calls into compiled model code.
Tensor layout	Often NHWC, because vision accelerators like contiguous pixel streams.	Often NCHW, the common ONNX/PyTorch-style layout, so image data may need transposition.
Data type	Usually <code>uint8</code> or <code>int8</code> ; quantization is part of the hardware contract.	Usually <code>float32</code> unless the model and compiler flow are explicitly quantized.
Data transfer	<code>input_vstreams[0].write(data)</code> pushes bytes into hardware stream/DMA channels.	<code>session.run([data])</code> converts NumPy arrays to internal OMTensors and calls compiled C entry points.

Conclusion

Both paths use the same AOT philosophy: the compiler resolves graph structure before deployment. Runtime code only injects tensors into a fast compiled artifact, not into a graph interpreter.

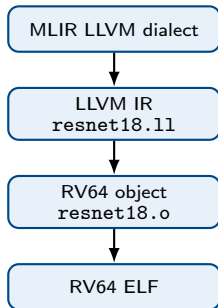
Engine vs Interface: Runtime Mental Model

Runtime	Analogy	Technical meaning
ONNX Runtime	A large factory engine. You give it the machine plan, the <code>.onnx</code> file. It loads the plan, prepares tools, schedules work, and runs the gears for every inference.	A full inference framework: graph loader, optimizer, scheduler, execution-provider dispatcher, kernel library, and session API.
ONNX-MLIR Runtime	A small funnel attached to a pre-molded metal block. The compiler already shaped the block, the <code>.so</code> , <code>.jar</code> , or executable. The runtime only feeds data into it cleanly.	A lightweight interface library: passes tensors between user code and compiled model code using structures such as <code>OMTensor</code> and <code>OMTensorList</code> .

For Gemmini

The Gemmini binary should behave like the pre-molded block: compiled code, constants, runtime wrappers, and hardware ABI calls are already prepared before execution.

LLVM Carries Typed Code Toward RV64



LLVM

Compiler layer that turns typed LLVM IR into target machine code.

- Every value in LLVM IR has a type.
- Examples: `i64`, `float`, `ptr`, `void`.
- Function calls must match declared argument types.
- `llc` emits RV64 object code.
- Target: `riscv64-unknown-linux-gnu`.

Without it: the compiler would not have a typed, target-independent handoff to produce RV64 object code. Runtime calls and data pointers would be much easier to miscompile.

How LLVM Types Protect Runtime Calls

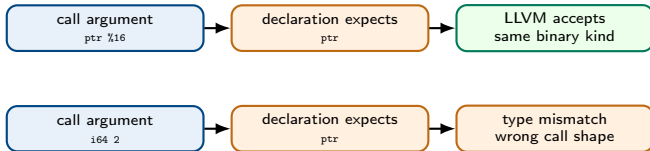
```
declare void @om_gemmini_conv_f32(  
    ptr, ptr, ptr, i64, i64)  
  
call void @om_gemmini_conv_f32(  
    ptr %16, ptr %25, ptr %34,  
    i64 2, i64 3)
```

Typed Means Explicit Shape of Values

LLVM does not see only names. It sees what binary kind each value has.

- `void`: function returns no value.
- `ptr`: address of an `OMTensor` descriptor.
- `i64`: 64-bit integer, used here for stride and padding.
- `float`: 32-bit floating-point data in constants.

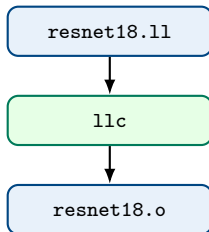
Why Typed Calls Matter



In This Project

LLVM types make the runtime boundary explicit: three tensor descriptor addresses, then two 64-bit convolution parameters.

llc Turns LLVM IR into RV64 Objects



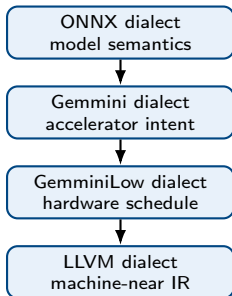
LLVM Static Compiler

llc lowers LLVM IR into target assembly or object code.

- Input: `resnet18.ll`.
- Output: relocatable RV64 object.
- Runtime symbols are resolved later by the linker.

Without it: `resnet18.ll` remains compiler IR. The model cannot become `resnet18.o`, so the linker cannot produce the final RV64 executable.

MLIR Keeps Multiple Abstraction Levels



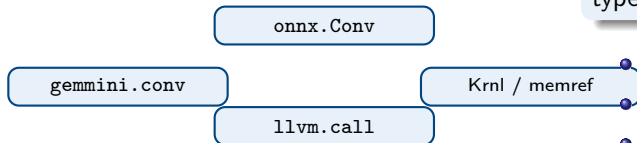
Multi-Level IR

MLIR represents the same program at multiple abstraction levels.

- High level: ONNX graph ops.
- Middle: accelerator intent.
- Low: hardware and binary details.

Without it: all compiler stages would need one flat representation. ONNX semantics, Gemmini intent, and hardware details would be mixed and difficult to verify or rewrite safely.

Dialects Give Each Level Its Own Language



Dialect

Namespace of MLIR operations, attributes, and types for one domain.

- ONNX: model semantics.
- Gemmini: accelerator semantics.
- GemminiLow: hardware actions.
- LLVM: machine-near calls and types.

Without it: operations would have no clear namespace or rules. A pass could confuse model-level `Conv` with hardware-level `load/execute/store` details.

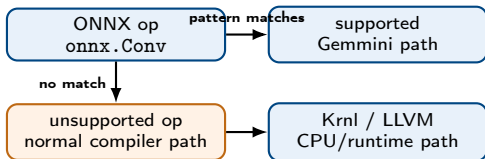
How Dialects Map to Pipeline Stages

Dialect / Layer	Example Meaning
ONNX	<code>onnx.Conv</code> : neural-network graph convolution.
Krnl	ONNX-MLIR kernel dialect: loops, <code>memref</code> buffers, allocations, constants, and runtime-call setup before LLVM.
Gemmini	Accelerator intent: this operation is selected for Gemmini.
GemminiLow	Hardware-level schedule: configure, load, execute, store.
LLVM dialect	MLIR representation close to LLVM IR.
LLVM IR	Textual LLVM form consumed by LLVM tools.
RV64 ELF	Final executable binary for Spike or hardware.

Conversion Chooses the Next Representation

Conversion

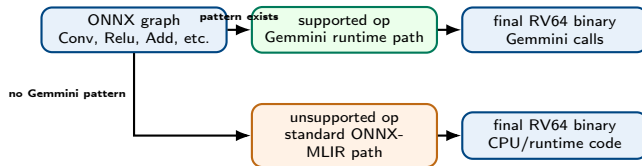
Tries to rewrite an operation when a valid pattern exists.



- A pattern says: this source op can become that target op.
- Legality checks shape, type, padding, stride, and attributes.
- If no safe Gemmini pattern exists, the op stays on the standard lowering path.

Without it: ONNX operations would stay in the wrong form. Supported Conv could not become Gemmini code, and unsupported ops could not be separated cleanly for fallback.

Fallback Preserves Correctness

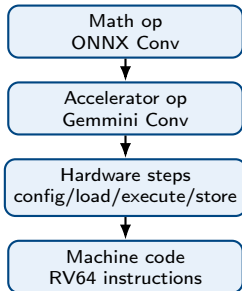


Meaning

Fallback means correctness first: unsupported operations are still compiled and executed, just not through Gemmini acceleration.

Without it: an unsupported operation might be forced into Gemmini incorrectly or dropped from the model. Fallback preserves correctness even when acceleration is not available.

Lowering Moves Closer to Execution



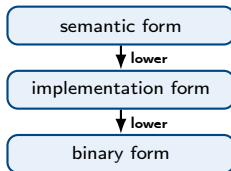
Lowering

Rewriting the same computation into a form with more execution details.

- Preserves behavior: same input tensors must produce the same output tensors.
- Exposes details: buffers, loops, calls, memory addresses, and hardware commands become visible.
- Moves closer to executable code: fewer model concepts, more instructions and symbols.

Without it: the model would remain too abstract for execution. A neural-network Conv would never become calls, buffers, RV64 instructions, or Gemmini commands.

What “Lowered” Means in Practice

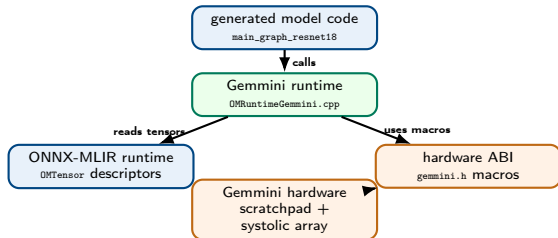


Lowered

Rewritten closer to the machine while preserving behavior.

- Conv becomes runtime setup/calls.
- Constants become global symbols.
- Runtime calls become RV64 calls.
- Gemmini helpers become custom instructions.

Runtime Bridges Generated Code to Hardware



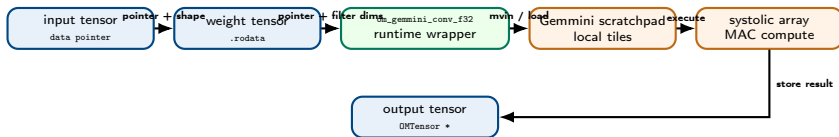
Runtime

Linked support code that executes between generated model code and low-level hardware access.

- Gives generated code stable C functions to call.
- Interprets OMTensor metadata.
- Chooses Gemmini helper kernels and data movement.
- Crosses into gemmini.h for custom instructions.

Without it: generated model code would have to program tensor layouts and Gemmini instructions directly. That would duplicate low-level logic and make the final executable fragile.

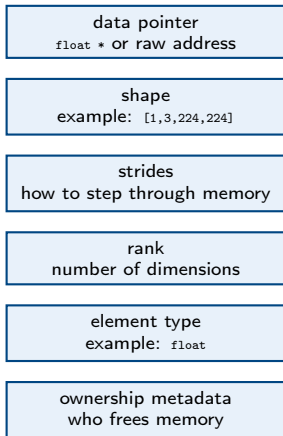
How Runtime Moves Tensor Data



Meaning

Runtime is the active bridge: it reads tensor descriptors, moves tiles to Gemmini-local memory, starts compute, and writes the result back into the output descriptor.

OMTensor Describes One Tensor



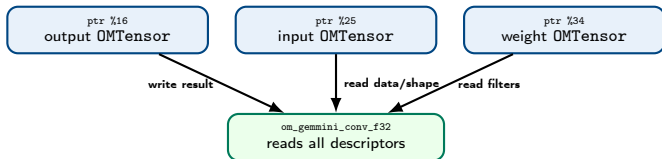
ONNX-MLIR Tensor Descriptor

OMTensor is not the tensor data itself. It is a descriptor that tells runtime code where the data is and how to interpret it.

- Input image: descriptor points to input floats.
- Weight tensor: descriptor points to constant weight memory.
- Output tensor: descriptor points to output memory.
- Gemmini runtime reads pointer, shape, rank, and type.

Without it: runtime would receive only raw addresses. It would not know shape, rank, element type, strides, or ownership, so convolution dimensions and memory access could be wrong.

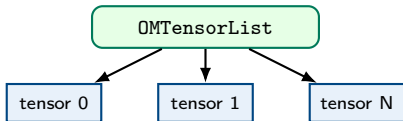
How Conv Receives OMTensor Descriptors



Concrete Meaning

The LLVM call passes addresses of descriptors. The runtime follows each descriptor to find the real tensor memory and dimensions.

OMTensorList Groups Model Inputs and Outputs



Input/Output Container

OMTensorList is the container used at model entry and model return.

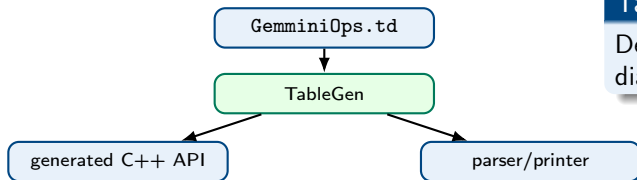
- Input list: one entry per model input.
- Output list: one entry per model output.
- Each entry is an OMTensor *.
- The graph function unpacks inputs and repacks outputs.

Without it: model entry and return would have no standard container for multiple tensors. Callers could pass inputs in the wrong order or lose output descriptors.

Types Used in This Pipeline

Type term	Concrete meaning in this project
<code>float</code>	32-bit floating-point number. ResNet weights and activations use this in the float test case.
<code>i64</code>	64-bit integer in LLVM IR. Used for values such as stride, padding, sizes, and dimensions.
<code>ptr</code>	LLVM IR pointer type. In the Conv call it points to an <code>OMTensor</code> descriptor.
<code>OMTensor *</code>	C/C++ pointer to one ONNX-MLIR tensor descriptor. Runtime uses it to find data and shape.
<code>OMTensorList *</code>	Pointer to a container holding multiple <code>OMTensor *</code> entries for model inputs or outputs.
<code>AnyType</code>	TableGen/MLIR placeholder meaning the operation accepts a broad MLIR type; later passes verify the real tensor type.

What Is a .td File?



TableGen Definition

Declarative source for MLIR operations and dialect metadata.

- Defines operands/results/traits.
- Generates C++ declarations.
- C++ files implement custom behavior.

TableGen Example: Concept

```
// Conceptual shape of a TableGen operation definition.
def Gemini_ConvOp : Gemini_Op<"conv", [...]> {
  let arguments = (ins AnyType:$input, AnyType:$weight);
  let results   = (outs AnyType:$output);
  let summary   = "High-level Gemini convolution";
}
```

- The `def` declares an operation.
- The operation belongs to a dialect.
- `AnyType` means TableGen allows a broad MLIR operand type here; verification/lowering decides the concrete tensor type later.
- Arguments, results, and attributes become part of the generated C++ API.

Source Tree Map

Connect each folder to its compiler responsibility before looking at individual files.

How the Repository Is Organized

Folder	Meaning
src/	Main compiler, dialect, accelerator, runtime, and support source.
include/	Public project headers.
docs/	Documentation and generated presentation material.
test/	ONNX-MLIR compiler and runtime tests.
gemmini/	Gemmini examples, scripts, tests, and generated artifacts.
gemmini_toolchain_build/	Local build tree for the Gemmini-enabled compiler.
third_party/	External dependencies managed separately from project source.

Why We Need a Backend Folder Map

What “Backend” Means Here

A backend is the part of the compiler that turns a general model representation into code for a specific execution target.

- Front-end: reads ONNX and creates MLIR operations.
- Middle-end: optimizes and rewrites operations.
- Backend: chooses Gemmini or fallback paths, lowers operations, emits runtime calls, and prepares target-specific code.

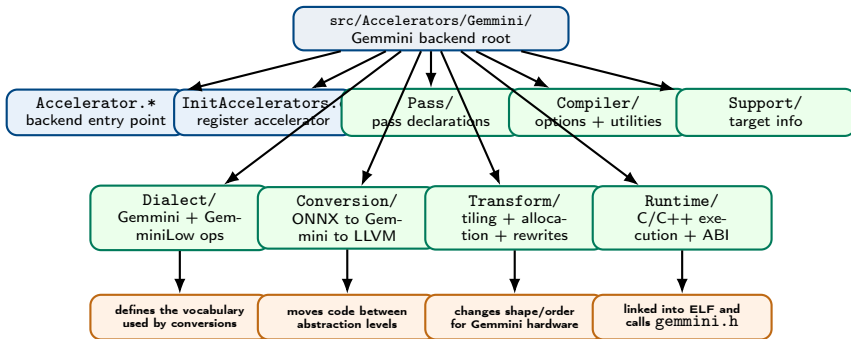
`src/Accelerators/Gemmini/`

What “Folder Map” Means

A folder map is a guide that connects source directories to their compiler responsibility.

- It is not only a file tree.
- It explains where each stage of the pipeline lives.
- It helps trace a feature from pass registration to lowering, runtime, ABI, and tests.

How the Gemmini Backend Is Organized



Read this map as responsibilities: registration and options at the top, compiler IR vocabulary and lowering in the middle, and runtime/ABI code at the hardware boundary.

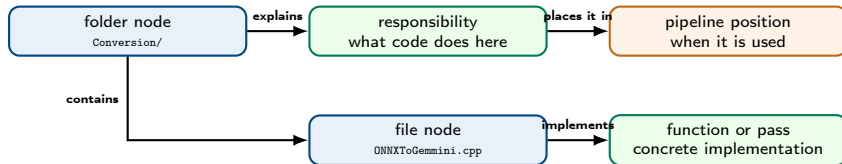
What the Backend Root Owns

Path	Responsibility
src/Accelerators/Gemmini/	Backend root: owns Gemmini compiler integration, dialects, conversions, transforms, runtime, target info, and pass declarations.
Accelerator.cpp/.hpp	Entry point that lets ONNX-MLIR recognize Gemmini as an accelerator backend.
InitAccelerators.cpp	Registration hook that makes Gemmini available to the accelerator registry.
src/Accelerators/Gemmini/.docs/	Local notes/documentation for the Gemmini backend implementation.
src/Accelerators/Gemmini/Pass/	Public pass declarations. Other compiler code includes these APIs to create and schedule Gemmini passes.

What Each Backend Subfolder Owns

Folder	What It Owns
<code>src/Accelerators/Gemmini/Compiler/</code> <code>src/Accelerators/Gemmini/Dialect/</code>	Compiler options and utilities: turns user flags into a concrete Gemmini pass pipeline. MLIR operation vocabulary: defines high-level <code>gemmini</code> operations and lower-level <code>gemminilow</code> operations.
<code>src/Accelerators/Gemmini/Conversion/</code>	Lowering between dialects: ONNX to Gemmini, Gemmini to GemminiLow, and GemminiLow to LLVM.
<code>src/Accelerators/Gemmini/Transform/</code>	Optimization and hardware fitting: tiling, static scratchpad allocation, and low-level rewrites.
<code>src/Accelerators/Gemmini/Runtime/</code>	Code linked into the final executable: C/C++ runtime functions plus Gemmini hardware ABI headers.
<code>src/Accelerators/Gemmini/Support/</code>	Target metadata: generated Gemmini hardware parameters used by compiler decisions.

How to Read Folder Graphs



Tree Graph Rule

Each branch answers one question: where is the code, what responsibility does it own, and when does it participate in the ONNX-to-RV64 pipeline.

Compiler Folder Connects Options to Passes

Location

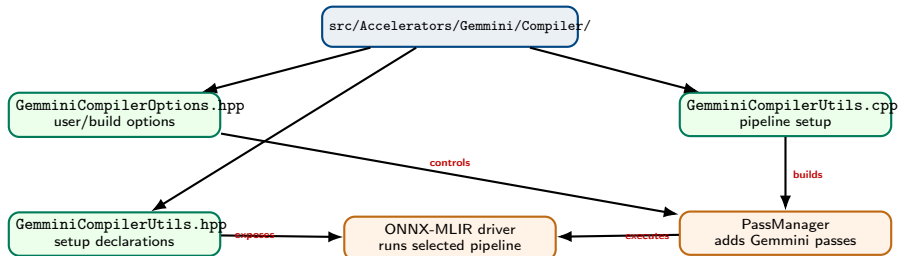
`src/Accelerators/Gemmini/Compiler/`

- `GemminiCompilerOptions.hpp`: user-facing compiler flags and options.
- `GemminiCompilerUtils.cpp`: pass-pipeline utility logic.
- `GemminiCompilerUtils.hpp`: declarations for Gemmini pipeline setup.

Purpose

This folder connects command-line or build-time Gemmini options to the actual compiler pass pipeline. It decides which Gemmini-specific passes should be added when the accelerator is enabled. It keeps pipeline setup code separate from the implementation of each conversion or transform pass. It gives the compiler one organized place to prepare ONNX-to-Gemmini lowering before code generation.

How Compiler Files Build the Pass Pipeline



controls: options decide which passes are enabled. **builds:** utility code assembles the pass pipeline. **exposes:** header gives other code callable declarations.

executes: driver runs the selected pass sequence.

Conversion Folder Moves Between Dialects

Location

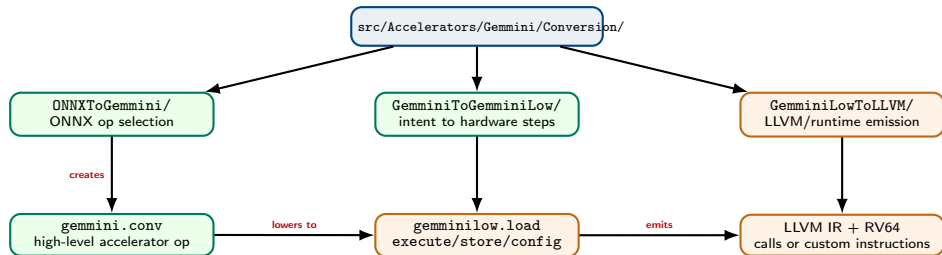
`src/Accelerators/Gemmini/Conversion/`

- `ONNXToGemmini/`: finds supported ONNX operations and maps them to Gemmini.
- `GemminiToGemminiLow/`: expands high-level Gemmini intent into lower-level hardware steps.
- `GemminiLowToLLVM/`: emits LLVM-level calls or inline assembly.

Purpose

This folder moves the model from high-level graph operations toward executable target code. It decides when an ONNX operation can become a Gemmini accelerator operation. It lowers abstract accelerator intent into explicit hardware-oriented load, execute, config, and store steps. It finally prepares those low-level operations for LLVM and the RISC-V backend.

How Conversion Files Lower the Model



creates: rewrites an ONNX operation into a Gemini operation. **lowers to:** replaces an abstract op with more concrete lower-level steps. **emits:** produces LLVM/runtime/RV64-facing code from GeminiLow.

Dialect Folder Defines Gemmini Vocabulary

Location

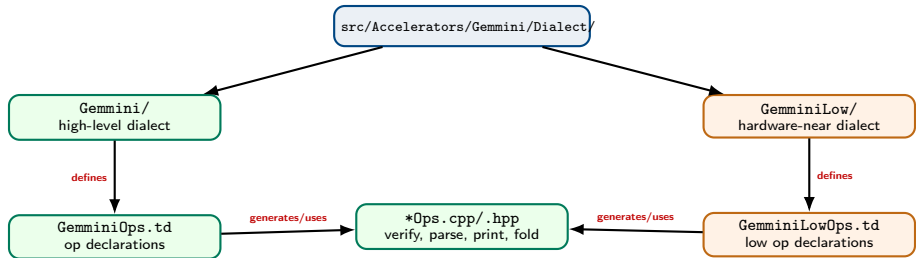
`src/Accelerators/Gemmini/Dialect/`

- `Dialect/Gemmini/`: high-level accelerator operations such as conv and matmul.
- `Dialect/GemminiLow/`: hardware-near operations such as load, execute, store, and config.
- `*.td`: declarative operation definitions.
- `*Ops.cpp`: operation behavior, side effects, folding, verification support.

Purpose

This folder defines the words that Gemmini passes are allowed to use inside MLIR. It separates high-level accelerator meaning from hardware-near scheduling details. It gives conversion and transform passes typed operations instead of loose strings or comments. It lets MLIR verify, print, parse, and rewrite Gemmini operations in a structured way.

How Dialect Files Define Operations



defines: TableGen files declare valid MLIR operations and fields. **generates/uses:** TableGen creates C++ operation scaffolding, while C++ files add behavior such as verification, parsing, printing, and folding.

Transform Folder Fits IR to Hardware

Location

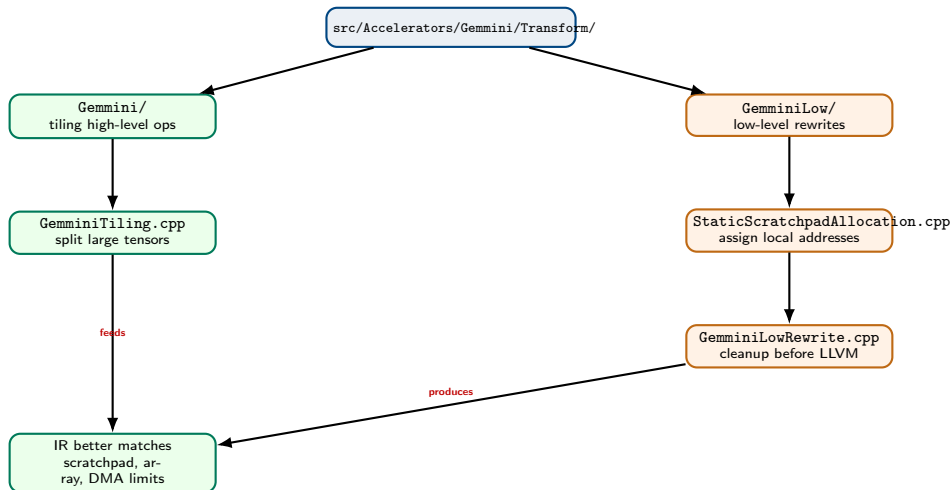
`src/Accelerators/Gemmini/Transform/`

- `Transform/Gemmini/GemminiTiling.cpp`: high-level tiling.
- `Transform/GemminiLow/StaticScratchpadAllocation.cpp`: scratchpad address assignment.
- `Transform/GemminiLow/GemminiLowRewrite.cpp`: low-level cleanup and rewrite patterns.

Purpose

This folder reshapes the compiler IR so it better matches Gemmini hardware limits. It tiles large tensors into blocks that can fit in scratchpad and accumulator memory. It assigns static local-memory locations so later lowering has concrete hardware addresses. It rewrites low-level operations to reduce ambiguity before LLVM code generation.

How Transform Files Prepare Hardware-Friendly IR



feeds: high-level tiling output becomes input for later low-level passes. **produces:** rewrite/allocation passes create IR that is closer to Gemmini hardware limits.

Runtime Folder Links Model Code to Gemini

Location

`src/Accelerators/Gemmini/Runtime/`

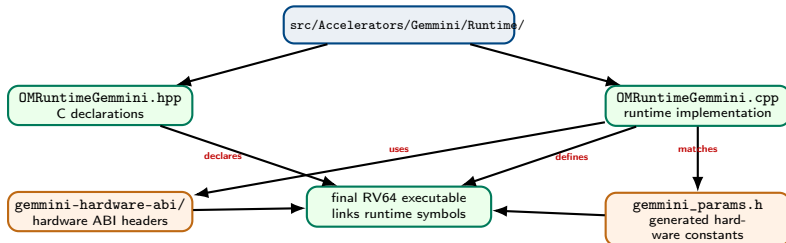
- `OMRuntimeGemmini.hpp`: declarations used by generated or lowered model code.
- `OMRuntimeGemmini.cpp`: C/C++ implementations linked into the final executable.
- `gemmini-hardware-abi/include/`: Gemini macros, parameters, and helper headers.
- This folder is where compiler output becomes callable code that can reach the accelerator.

Purpose

This folder provides the executable software bridge between ONNX-MLIR generated code and Gemini hardware.

It implements stable runtime functions such as convolution wrappers that the lowered model can call. It includes the hardware ABI headers that encode Gemini custom instruction usage and parameters. It keeps model code, runtime code, and hardware commands connected during final linking and execution.

How Runtime Files Reach the ABI



declares: header publishes runtime function signatures. **defines:** C++ file implements those functions. **uses:** runtime calls ABI macros. **matches:** runtime constants align with generated Gemini parameters.

Support Folder Shares Hardware Facts

Location

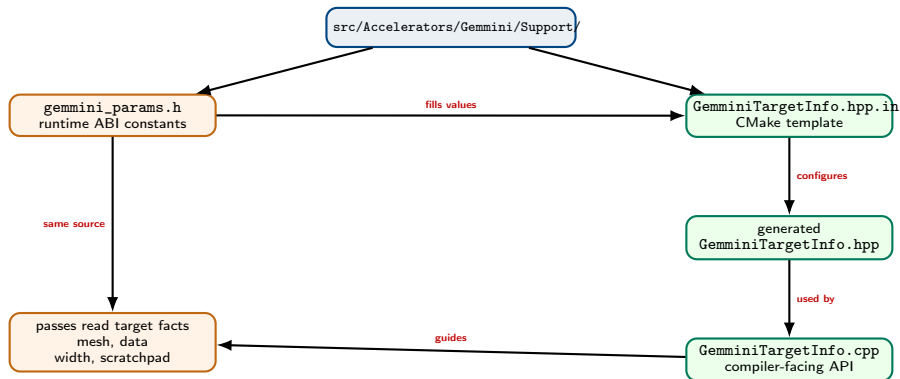
`src/Accelerators/Gemmini/Support/`

- `GemminiTargetInfo.hpp.in`: CMake template generated from `gemmini_params.h`.
- `GemminiTargetInfo.cpp`: target information implementation.
- Important reason: the compiler must know hardware sizes such as mesh dimensions, scratchpad capacity, and data widths before lowering.
- Generated information keeps compiler assumptions aligned with the runtime ABI and `gemmini_params.h`.

Purpose

This folder keeps target-specific hardware facts available to compiler code. It prevents the compiler from hard-coding a second copy of Gemmini configuration values. It lets CMake generate target information from the same parameters used by the runtime ABI. It helps passes make lowering decisions that match the actual configured Gemmini hardware.

How Support Files Avoid Duplicate Configuration



fills values: hardware constants are copied into the template. **configures:** CMake produces the generated target-info header. **used by:** implementation reads the generated declarations. **guides:** passes use target facts for lowering decisions.

Pass Folder Exposes Compiler Steps

Location

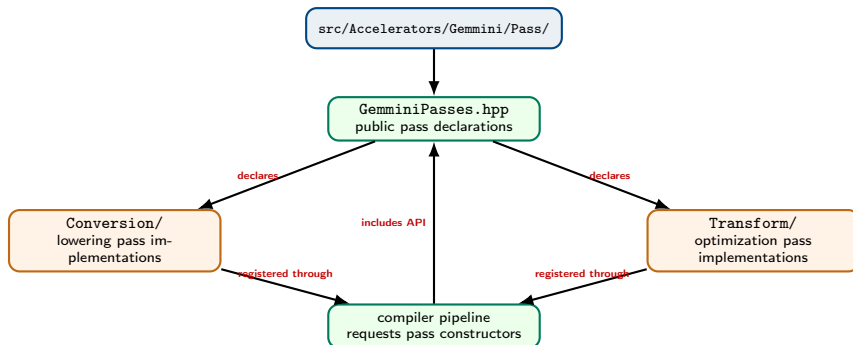
`src/Accelerators/Gemmini/Pass/`

- Main role: declare Gemmini pass creation APIs in one public place.
- `GemminiPasses.hpp`: exposes pass constructors used by the compiler pipeline.
- Pass declarations are not the full implementations; implementations live in `Conversion/` and `Transform/`.
- This folder is the connector between pipeline setup and concrete lowering/optimization code.
- Example: a driver can request “convert ONNX to Gemmini” without knowing the internal file that implements the rewrite patterns.

Purpose

This folder gives the rest of ONNX-MLIR a clean way to ask for Gemmini compiler passes. It exposes pass creation functions without forcing callers to know implementation files. It keeps the public pass interface separate from conversion and transform internals. It helps the compiler pipeline schedule Gemmini lowering steps in the correct order.

How Pass Files Register Lowering Work



declares: public header names pass creation functions. **includes API:** pipeline code imports those declarations. **registered through:** concrete conversion/-transform passes are made available to the pipeline by this public interface.

Compiler Architecture

Follow the model as compiler passes lower it from ONNX semantics to Gemmini-aware execution.

Use the Real ONNX Folder Layout

Actual Location

`src/Dialect/ONNX/`

- The actual tree contains `ONNXOps/`, `ElementsAttr/`, and `Transforms/`.
- There is no `src/Dialect/ONNX/ShapeInference/` folder in this checkout.
- Shape-related logic is organized through ONNX operation implementations, generated code, and transform utilities.
- The presentation therefore uses the real folder name: `Transforms/`.

ONNX Stage Preserves Model Semantics

Role

Represent the user model in MLIR while preserving ONNX semantics.

- Input file: `model.onnx`.
- Important source folder: `src/Dialect/ONNX/`.
- Main operation definitions: `ONNXOps.td` and operation-specific files under `ONNXOps/`.
- Output: ONNX dialect MLIR that can be transformed by ONNX-MLIR passes.

Location

`src/Accelerators/Gemmini/Conversion/ONNXToGemmini/`

- Checks whether an ONNX operation is supported by the Gemmini backend.
- Converts selected operations such as Conv, MatMul, Gemm, Add, Relu, and pooling.
- Keeps unsupported operations available for fallback or other lowering paths.
- This is the first accelerator-specific decision point.

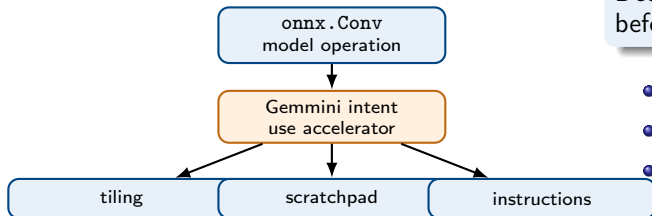
Gemmini Dialect Captures Accelerator Intent

Why it exists

The compiler needs a representation that says "this operation should use Gemmini" without committing to every hardware instruction yet.

- A `gemmini.conv` still looks like a convolution.
- The operation can carry shape, stride, padding, and dataflow information.
- Tiling and legality checks can still operate at a meaningful neural-network level.
- This level is easier to reason about than raw DMA and custom instructions.

Accelerator Intent Marks Work for Gemini



Accelerator Intent

Decision that an operation should use Gemini before exact hardware steps are chosen.

- Keeps Conv/MatMul semantics.
- Still carries shape, stride, padding.
- Later passes choose schedule and instructions.

Without it: the compiler would jump from ONNX directly to raw hardware details. Tiling, legality checks, and accelerator selection would be harder to reason about.

GemminiLow Exposes Hardware Steps

Why it exists

Gemmini hardware execution is not one abstract operation. It needs configuration, memory transfers, compute commands, and stores.

- `gemminilow.config`: program dataflow, strides, scaling, and mode.
- `gemminilow.load`: move data into the scratchpad.
- `gemminilow.execute`: launch systolic-array computation.
- `gemminilow.store`: move results back to memory.

Without it: one high-level `gemmini.conv` would hide the required hardware sequence. The compiler could not schedule loads, compute, stores, or scratchpad use explicitly.

How Gemmini Becomes GemminiLow

```
// High-level intent:
gemmini.conv(input, weight, output, stride=2, pad=3)

// Lower-level execution shape:
gemminilow.config(dataflow, strides, scale)
gemminilow.load(input_tile, scratchpad_A)
gemminilow.load(weight_tile, scratchpad_B)
gemminilow.execute(scratchpad_A, scratchpad_B, accumulator)
gemminilow.store(accumulator, output_tile)
```

- Gemmini is compiler-friendly.
- GemminiLow is hardware-scheduling-friendly.

Tiling Makes Large Tensors Fit

Location

`src/Accelerators/Gemmini/Transform/Gemmini/GemminiTiling.cpp`

- Large tensors may not fit directly into accelerator-local storage.
- Tiling partitions the work into blocks that match Gemmini array and scratchpad limits.
- Tiling exposes reuse: weights or activations can stay in local memory across multiple operations.
- The result is a better schedule for later GemminiLow lowering.

Without it: large ResNet tensors may exceed scratchpad or accumulator limits. The runtime would move too much data at once or lose reuse opportunities.

Scratchpad Allocation Assigns Local Addresses

Location

`src/Accelerators/Gemmini/Transform/GemminiLow/StaticScratchpadAllocation.cpp`

- Gemmini has a scratchpad and accumulator memory separate from normal DRAM.
- The compiler must assign addresses for tiles and intermediate results.
- Static allocation makes those addresses explicit before LLVM lowering.
- This reduces ambiguity when emitting low-level code.

Without it: GemminiLow operations would refer to local memory without stable addresses. Tiles could overlap, overwrite each other, or be loaded from the wrong scratchpad location.

GemminiLow to LLVM Emits Runtime/Instruction Hooks

Location

`src/Accelerators/Gemmini/Conversion/GemminiLowToLLVM/`

- Converts GemminiLow operations into LLVM-compatible constructs.
- Can emit calls to runtime functions.
- Can emit inline assembly for RISC-V custom instructions.
- The output is ready for LLVM IR generation and RISC-V code generation.

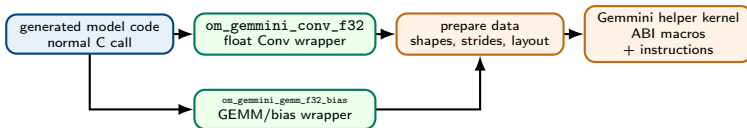
Hardware ABI Headers Define Gemini Commands

Location

`src/Accelerators/Gemini/Runtime/gemmini-hardware-abi/include/`

- `gemmini.h`: main instruction macros and low-level accelerator interface.
- `gemmini_params.h`: generated hardware configuration.
- `gemmini_nn.h`: neural-network helper functions.
- `gemmini_counter.h`: performance counters.
- These files define the contract expected by generated and runtime code.

Runtime Functions Execute Selected Ops



Location

`src/Accelerators/Gemmini/Runtime/OMRuntimeGemmini.cpp`

Which C++ / HPP Files Make the Path Work

File	Role in the ResNet-18 Gemmini path
GemminiCompilerUtils.cpp	Adds Gemmini conversion passes to the pass manager.
ONNXToGemmini.cpp	Matches supported ONNX Conv and creates Gemmini runtime calls.
ONNXToGemmini.hpp	Declares the conversion pass and pattern population API.
OMRuntimeGemmini.hpp	Declares om_gemmini_conv_f32.
OMRuntimeGemmini.cpp	Implements om_gemmini_conv_f32 and the shared Conv implementation.
gemmini.h	Encodes Gemmini hardware ABI macros and custom instructions.

How the Compiler Adds the Gemini Pass

```
// src/Accelerators/Gemmini/Compiler/GemminiCompilerUtils.cpp  
pm.addNestedPass<func::FuncOp>(createONNXToGemminiPass());
```

- This is where the Gemini lowering pass is inserted into the compiler pipeline.
- Without this hook, the model would not be rewritten to Gemini runtime calls.

How ONNX Conv Converts to Gemmini Runtime

```
// src/Accelerators/Gemmini/Conversion/ONNXToGemmini/ONNXToGemmini.cpp
struct ONNXConvToGemminiLowering
: public OpConversionPattern<ONNXConvOp> {
    LogicalResult matchAndRewrite(ONNXConvOp op,
        ONNXConvOpAdaptor adaptor,
        ConversionPatternRewriter &rewriter) const final {
        if (!isSupportedGemminiConvF32(op))
            return failure();
        ...
        StringRef funcName = "om_gemmini_conv_f32";
    }
};
```

- The pattern recognizes supported ONNX Conv operations.
- It chooses the runtime function name used later in LLVM IR.

How Generated Code Sees the Runtime Function

```
// src/Accelerators/Gemmini/Runtime/OMRuntimeGemmini.hpp  
void om_gemmini_conv_f32(OMTensor *out, const OMTensor *x,  
    const OMTensor *w, int64_t stride, int64_t pad);
```

- Generated code can call this function through the C ABI.
- Arguments are tensor descriptors plus convolution attributes.

How the Runtime Implements Conv

```
// src/Accelerators/Gemmini/Runtime/OMRuntimeGemmini.cpp
static void om_gemmini_conv_f32_impl(OMTensor *out,
    const OMTensor *x, const OMTensor *w,
    const OMTensor *b, int64_t stride, int64_t pad);

void om_gemmini_conv_f32(OMTensor *out, const OMTensor *x,
    const OMTensor *w, int64_t stride, int64_t pad) {
    om_gemmini_conv_f32_impl(out, x, w, NULL, stride, pad);
}
```

- Public wrapper calls the shared no-bias/bias implementation.
- The implementation prepares data and calls Gemmini kernels.

Execution Example

Track one ResNet-18 Conv from ONNX model files to RV64 disassembly.

Where the ResNet-18 Example Lives

Local Test

`gemmini/tests/15-resnet18-float/`

- Contains generated MLIR, LLVM IR, RV64 object, linked ELF, and disassembly files.
- Used to compare Gemmini and non-Gemmini inference flows.
- Main binary: `resnet18_rv64`.
- Main disassembly: `resnet18_rv64.disasm`.

How the ResNet-18 Script Runs the Pipeline

```
bash "$ROOT_DIR/gemmini/examples/tools/run_float_model_spike.sh" \  
  --model "$MODEL_DIR/resnet18.onnx" \  
  --out-dir "$TEST_DIR" \  
  --no-sim
```

- The test compiles and links the model without running Spike by default.
- The script writes artifacts into `gemmini/tests/15-resnet18-float`.

How ResNet-18 Artifacts Chain Together

- 1 `resnet18_onnxir.onnx.mlir`: ONNX dialect inspection output.
- 2 `resnet18_krnl.onnx.mlir`: lowered MLIR/Krnl-oriented output.
- 3 `resnet18.onnx.mlir`: LLVM dialect MLIR.
- 4 `resnet18.ll`: LLVM IR.
- 5 `resnet18.o`: RV64 object file.
- 6 `resnet18_rv64`: statically linked RV64 ELF executable.
- 7 `resnet18_rv64.disasm`: disassembled executable.

Where Conv Starts in the ONNX Model

File

`gemmini/examples/15-resnet18-float/resnet18.onnx`

Local ONNX inspection shows:

- Model nodes: 70.
- First node: `Conv resnetv15_conv0_fwd`.
- First Conv inputs: `data`, `resnetv15_conv0_weight`.
- First weight initializer: `resnetv15_conv0_weight [64,3,7,7]`.
- Initializers: 102.

Why Weight Names Change During Lowering

- ONNX uses graph names such as `resnetv15_conv0_weight`.
- ONNX-MLIR may fold, fuse, reorder, or rename constants during lowering.
- Later artifacts use compiler-generated symbols such as `constant_30_resnet18`.
- Therefore, tracking weights means following both semantic role and emitted symbol names.

Example

The disassembly uses `constant_30_resnet18`; the original ONNX model uses named initializers such as `resnetv15_conv0_weight`.

How Krnl Turns Weights into Compiler Globals

```
// gemmini/tests/15-resnet18-float/resnet18_krnl.onnx.mlir
%0 = "krnl.global"() <{
  name = "constant_0",
  shape = [512, 1, 1],
  value = dense<...> : tensor<512x1x1xf32>
}> : () -> memref<512x1x1xf32>
```

- Krnl MLIR turns ONNX initializers into compiler-managed global constants.
- This stage is still MLIR, not a binary file.

How LLVM Dialect Carries Weight Symbols

```
// gemmini/tests/15-resnet18-float/resnet18.onnx.mlir
llvm.mlir.global internal constant @constant_30_resnet18(
  dense<[[[1.30345976]], [[0.482770979]], ...]>
  : tensor<64x1x1xf32>
) {alignment = 16 : i64}

%47 = llvm.mlir.addressof @constant_30_resnet18
```

- The constant is now an LLVM-dialect global.
- The model code takes its address to build an `OMTensor` for weights.

How LLVM IR Declares Runtime and Constants

```
// gemmini/tests/15-resnet18-float/resnet18.ll
@constant_30_resnet18 = internal constant
    [64 x [1 x [1 x float]]] ...

declare void @om_gemmini_conv_f32(
    ptr, ptr, ptr, i64, i64)
```

- The weight is now an LLVM IR global.
- The Gemmini convolution function is declared but not implemented in this file.

How LLVM IR Calls Gemmini Conv

```
store i64 64, ptr %35, align 8
store i64 3, ptr %37, align 8
store i64 7, ptr %39, align 8
store i64 7, ptr %41, align 8

call void @om_gemmini_conv_f32(
  ptr %16, ptr %25, ptr %34, i64 2, i64 3)
```

- This call comes from `resnet18.11`.
- The first convolution uses 64 filters, 3 channels, 7x7 kernel, stride 2, padding 3.

What the LLVM IR Conv Call Transfers

- The generated code builds `OMTensor` descriptors for input, output, and weights.
- The weight tensor data pointer is set to the constant global.
- The output tensor points to allocated model memory.
- The runtime call transfers control from generated model code into Gemmini runtime code.
- Data movement into Gemmini scratchpad happens inside runtime/kernel code, not in the LLVM call line itself.

How to Read the Conv Call Arguments

- `ptr %16`: address of the output `OMTensor`; runtime writes result metadata/data through it.
- `ptr %25`: address of the input `OMTensor`; runtime reads input shape and data pointer.
- `ptr %34`: address of the weight `OMTensor`; runtime reads filter shape and constant-data pointer.
- `i64 2`: 64-bit integer value for convolution stride.
- `i64 3`: 64-bit integer value for convolution padding.
- The compiled graph calls a Gemini runtime function instead of emitting a plain CPU convolution loop.

How RV64 Disassembly Calls Runtime

```
11474: 00196637 lui    a2,0x196
11478: 3d060613 addi   a2,a2,976
          # 1963d0 <constant_30_resnet18>
...
114cc: 4689    c.li   a3,2
114ce: 470d    c.li   a4,3
114d6: 0000b097 auipc  ra,0xb
114da: 898080e7 jalr   ra,-1896(ra)
          # 1bd6e <om_gemmini_conv_f32>
```

- The linked binary calls `om_gemmini_conv_f32`.
- `constant_30_resnet18` is the first convolution weight block.

What the Object File Knows Before Linking

```
$ nm -n -S resnet18.o
                 U om_gemmini_conv_f32
0000000000000000 0000000000003750 T main_graph_resnet18
0000000000003750 0000000000000064 T _mlir_ciface_main_graph_resnet18
00000000000037b4 0000000000000220 T run_main_graph_resnet18
0000000002c8ec00 0000000000000100 r constant_30_resnet18
```

- U means the function is unresolved in the object.
- The model object contains the graph code and read-only constants.

What the ELF Contains After Linking

```
$ nm -n -S resnet18_rv64
0000000000011bea T main_graph_resnet18
000000000001539e T run_main_graph_resnet18
00000000000174c2 t om_gemmini_conv_f32_impl
000000000001a078 T om_gemmini_conv_f32
00000000002e24b70 r constant_30_resnet18
```

- The linker resolves `om_gemmini_conv_f32` from the Gemmini runtime archive.
- Constants move into the final executable `.rodata` section.

Where Weights Live in Read-Only Data

```
resnet18.o:  
  .rodata offset 0x003b20 size 0x2c96e18 align 16  
  
resnet18_rv64:  
  .rodata address 0x18b640 offset 0x17b640  
  size 0x2cbe900 align 16
```

- Model weights are stored in read-only data.
- The object file has section offsets; the linked ELF has final virtual addresses.

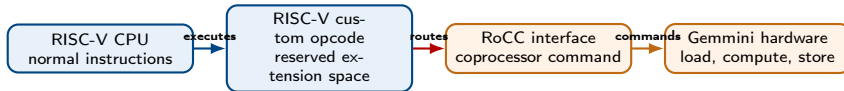
How to Read the Disassembly

- The compiler has already selected runtime helper calls before linking.
- The linker resolves `om_gemmini_conv_f32` to an address in the final RV64 executable.
- The disassembly shows normal RISC-V setup instructions followed by a function call.
- Inside the runtime and Gemmini kernels, custom accelerator instructions are emitted.

How `om_gemmini_conv_f32` Appears Across Stages

File	Evidence
<code>resnet18.onnx</code>	First Conv node and weight initializer exist in the ONNX graph.
<code>resnet18_krn1.onnx.mlir</code>	Constants represented through <code>krnl.global</code> .
<code>resnet18.onnx.mlir</code>	LLVM dialect declares and calls <code>om_gemmini_conv_f32</code> .
<code>resnet18.ll</code>	LLVM IR contains <code>declare void @om_gemmini_conv_f32</code> and calls it.
<code>resnet18.o</code>	Symbol is unresolved; model code and constants are present.
<code>resnet18_rv64</code>	Runtime symbol is linked into the final ELF.
<code>resnet18_rv64.disasm</code>	Disassembly shows calls and Gemini custom <code>.insn</code> instructions.

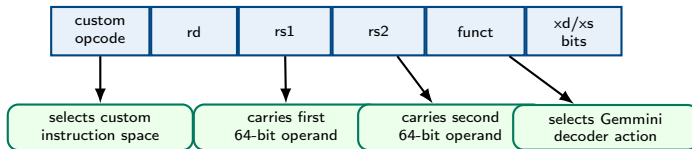
Why Gemini Uses Custom Instructions



Term	Meaning in this project
Instruction	A binary command decoded by the RISC-V core. Normal instructions do CPU work; custom instructions can command an attached accelerator.
Custom instruction	A RISC-V instruction using custom opcode space. The CPU recognizes that it belongs to a coprocessor path instead of a standard ALU/load/store path.
RoCC	Rocket Custom Coprocessor interface. It carries command fields from the CPU pipeline to Gemini.
Gemmini command	The hardware meaning attached to a custom instruction, such as configure, move input, compute, or move output.

`src/Accelerators/Gemmini/Runtime/gemmini-hardware-abi/include/gemmini.h`

How a Gemini Instruction Is Packed



In `gemmini.h`, most commands use `ROCC_INSTRUCTION_RS1_RS2(XCUSTOM_ACC, rs1, rs2, funct)`. The C macro packs normal C values into registers, and the inline assembly emits a RISC-V custom instruction.

Why this matters

The same binary instruction is understood by three sides: C code knows the macro arguments, the RISC-V core knows how to issue the custom opcode, and Gemini knows the `funct` command number.

Macro Path: C Helper to RoCC Instruction

```
#define ROCC_INSTRUCTION_RS1_RS2(x, rs1, rs2, funct) \  
    ROCC_INSTRUCTION_0_R_R(x, rs1, rs2, funct)  
  
#define gemmini_extended_mvin(dram_addr, spad_addr, cols, rows) \  
    ROCC_INSTRUCTION_RS1_RS2(XCUSTOM_ACC, dram_addr, \  
        ((uint64_t)(rows) << (ADDR_LEN + 16)) | \  
        ((uint64_t)(cols) << ADDR_LEN) | (spad_addr), k_MVIN)
```

Field	Explanation
XCUSTOM_ACC	Selects the accelerator custom instruction slot used for Gemmini.
dram_addr	First source register: host/DRAM address of the tensor tile.
rows, cols, spad_addr	Packed into the second source register as tile shape plus scratchpad destination.
k_MVIN	Function code telling Gemmini this command is a memory move into scratchpad.

src/Accelerators/Gemmini/Runtime/gemmini-hardware-abi/include/gemmini.h

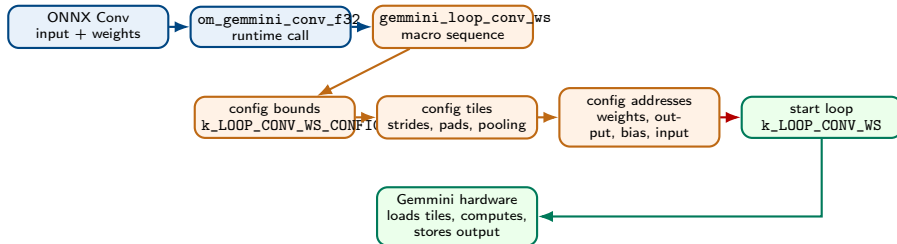
Instruction Families Used by Gemmini

Family	Example function codes	Purpose
Configure	k_CONFIG, CONFIG_EX, CONFIG_LD, CONFIG_ST	Program execution mode, dataflow, activation, strides, load scaling, and store behavior.
Move in	k_MVIN, k_MVIN2, k_MVIN3	Move input, weights, or bias tiles from DRAM into Gemmini scratchpad or accumulator memory.
Compute	k_PRELOAD, k_COMPUTE_PRELOADED,	Feed matrix tiles into the systolic array and accumulate partial results.
Move out	k_COMPUTE_ACCUMULATE k_MVOUT, k_MVOUT_SPAD	Copy completed output tiles from Gemmini memory back to DRAM.
Hardware loops	k_LOOP_WS, k_LOOP_CONV_WS	Let Gemmini run larger matmul/conv loops after receiving packed bounds, addresses, and strides.

Simple rule

`k_*` values are not CPU functions. They are command IDs decoded by Gemmini hardware after the RoCC instruction reaches the accelerator.

How Conv Becomes Several Custom Instructions



One ONNX Conv is larger than one hardware instruction. The runtime expands it into a command sequence that gives Gemini dimensions, addresses, padding, strides, activation, and launch information.

Conv Loop Macro Sends Configuration Commands

```
#define gemmini_loop_conv_ws(..., weights, output, bias, input, ...) \  
{ \  
    ROCC_INSTRUCTION_RS1_RS2(XCUSTOM_ACC, packed_dims_1, packed_dims_2, \  
        k_LOOP_CONV_WS_CONFIG_1) \  
    ROCC_INSTRUCTION_RS1_RS2(XCUSTOM_ACC, packed_tile_1, packed_tile_2, \  
        k_LOOP_CONV_WS_CONFIG_2) \  
    ROCC_INSTRUCTION_RS1_RS2(XCUSTOM_ACC, weights, output, \  
        k_LOOP_CONV_WS_CONFIG_5) \  
    ROCC_INSTRUCTION_RS1_RS2(XCUSTOM_ACC, bias, input, \  
        k_LOOP_CONV_WS_CONFIG_6) \  
    ROCC_INSTRUCTION_RS1_RS2(XCUSTOM_ACC, packed_flags_1, packed_flags_2, \  
        k_LOOP_CONV_WS) \  
}
```

The real macro has many parameters because convolution needs many facts: tensor sizes, kernel size, padding, stride, pooling, weight address, input address, bias address, output address, activation, and layout flags.

src/Accelerators/Gemmini/Runtime/gemmini-hardware-abi/include/gemmini.h

How to Read `.insn` in Disassembly

Disassembly part	Meaning
Address	The left address, for example 181be, is where this instruction lives in the linked RV64 executable.
Hex word	The middle value, for example 00e8b07b, is the raw 32-bit instruction encoding.
<code>.insn</code>	The disassembler does not know a nice Gemini mnemonic, so it prints the generic custom-instruction form.
Runtime location	These instructions usually appear inside linked Gemini runtime/helper code, not directly beside the original ONNX node text.
Proof of hardware path	Seeing <code>.insn</code> confirms the binary contains RoCC/Gemini commands, not only CPU fallback code.

Important

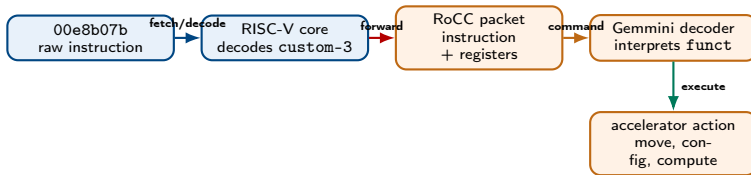
`.insn` is not an error. It is the assembler/disassembler's generic way to represent an instruction whose exact accelerator meaning is defined by the Gemini ABI and hardware decoder.

Main idea

This `.insn` line is not a mistake and not a generic placeholder. It is a custom RISC-V instruction that the RISC-V core forwards through RoCC to the Gemmini accelerator.

Field	What it means
0x7b opcode	The low 7 bits are 0x7b, which is RISC-V custom-3. RISC-V reserves custom opcode spaces such as 0x0b custom-0, 0x2b custom-1, 0x5b custom-2, and 0x7b custom-3 so extensions do not collide with standard instructions.
funct7	The high command field selects the Gemmini decoder action. In Gemmini, these values distinguish configuration, data movement, compute, loop setup, and other accelerator commands.
Register fields	The middle bits encode <code>rd</code> , <code>rs1</code> , and <code>rs2</code> . The registers carry values such as DRAM addresses, scratchpad addresses, packed tile sizes, strides, or command flags.
Execution	The CPU does not treat this like a normal ALU instruction. It issues a RoCC command packet containing the instruction and the selected register values. Gemmini decodes that packet and performs the low-level accelerator operation.

How 00e8b07b Reaches Gemini



The line is machine code for telling Gemini to perform one low-level hardware operation. A complete convolution or matrix multiplication is built from several such commands, plus normal RISC-V instructions and runtime control code.

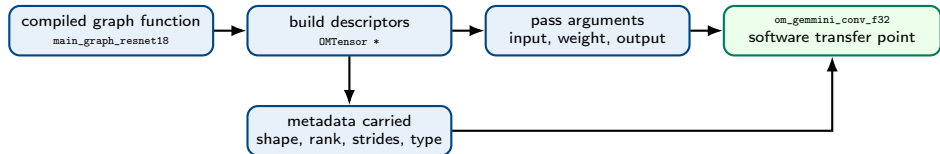
`src/Accelerators/Gemmini/Runtime/gemmini-hardware-abi/rocc-software/src/xcustom.h`

Example: Gemmini Custom Instructions

```
181be: 00e8b07b .insn 4, 0x00e8b07b
18234: 04e5b07b .insn 4, 0x04e5b07b
18262: 00f7307b .insn 4, 0x00f7307b
182cc: 05e8307b .insn 4, 0x05e8307b
182fc: 04f8307b .insn 4, 0x04f8307b
```

- These lines appear in `resnet18_rv64.disasm`.
- The generic disassembler prints raw custom RISC-V encodings as `.insn`.
- They correspond to Gemmini RoCC operations emitted by the ABI/runtime path.

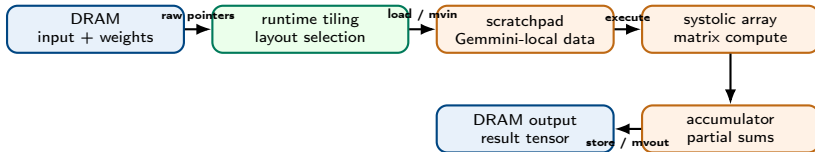
How Generated Code Transfers Tensors to Runtime



Meaning

The generated code does not directly program Gemini. It hands tensor descriptors to a runtime function with a stable C ABI.

How Runtime Transfers Data to Gemini Hardware



Meaning

Runtime turns model-level tensors into hardware-level transfers: DRAM to scratchpad, scratchpad to compute, accumulator back to DRAM.

Why Transfer Cost Matters

- Gemini acceleration is only useful if transfer overhead is controlled.
- Tiling decides how much data is moved at once.
- Scratchpad reuse reduces repeated DRAM traffic.
- Weight constants live in ELF `.rodata`; at runtime they are read through pointers and moved into Gemini-local storage.

How Conv Moves from ONNX to Binary

- 1 ONNX model contains a Conv node and constant weight tensors.
- 2 ONNX-MLIR imports the graph into ONNX dialect MLIR.
- 3 Gemmini conversion chooses supported Conv operations.
- 4 Lowering produces runtime calls such as `om_gemmini_conv_f32`.
- 5 LLVM produces RV64 object code.
- 6 Linker combines model object, Gemmini runtime archive, C runtime archive, and runner.
- 7 Disassembly shows calls and raw Gemmini custom instructions.

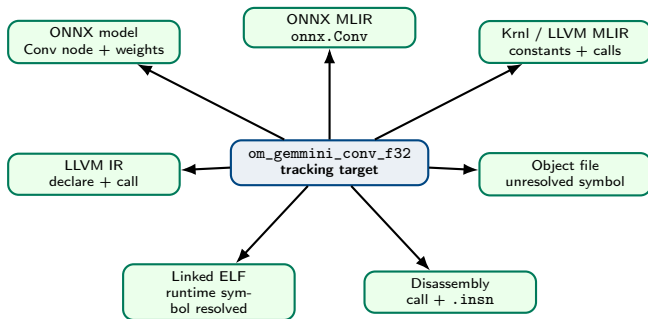
Why Runtime Calls Appear Before Raw Instructions

- A model-level Conv involves tensor descriptors, shapes, strides, padding, allocation, and cleanup.
- It is more maintainable to call a runtime wrapper for complex setup.
- The runtime can implement tiling, packing, quantization, and Gemmini kernel calls.
- The low-level Gemmini kernel eventually emits custom instructions through the ABI macros.

Validation and Takeaways

Use the artifact chain to debug, validate, and explain the pipeline.

Troubleshooting Map: Where Did Conv Go?



Debug rule

If Conv disappears, check the first missing artifact: unsupported ONNX pattern, missing runtime declaration, link failure, or ABI/custom-instruction path.

Validation Checklist

- Check the MLIR files after each important pass.
- Confirm runtime calls in `resnet18.ll`.
- Confirm the final ELF is RV64 and statically linked as expected.
- Confirm disassembly includes `om_gemmini_conv_f32` calls.
- Confirm Gemmini custom instructions appear as `.insn` in disassembly.
- Compare Spike timing JSON outputs for Gemmini vs non-Gemmini.

Key Takeaways

Takeaway	What to remember
Compiler lowers meaning Runtime bridges tensors ABI protects binary agreement GemminiLow exposes hardware Artifacts prove the path	ONNX starts as graph semantics and becomes Gemmini/LLVM/RV64 execution details. Generated code calls stable functions that understand <code>OMTensor</code> and <code>OMTensorList</code> . C calls and Gemmini hardware commands work only when caller and callee agree. Config, load, execute, and store become visible before final LLVM/RV64 lowering. Follow Conv through ONNX, MLIR, LLVM IR, object, ELF, and disassembly.

What This Pipeline Achieves

- ONNX-MLIR supplies the multi-level compiler infrastructure.
- Gemmini adds accelerator-specific dialects, conversion passes, runtime code, and ABI integration.
- Gemmini is the high-level accelerator dialect; GemminiLow is the hardware-oriented dialect.
- The ResNet-18 example demonstrates the full path from model artifacts to RV64 ELF and disassembly.

Project: Gemmini Pipeline in ONNX-MLIR

Company: Siliconesignal Technologies

July 15, 2026